

数值计算方法

对称矩阵特征值的计算方法

信息管理与工程学院
冯银波

带原点位移的QR迭代法

之前讲的QR迭代法收敛速度是线性的，收敛速度取决于特征值的分离程度。下面考虑QR迭代法的加速。

设 $H^{(0)}$ 是由初始矩阵 A 上Hessenberg化后得到的矩阵， $H^{(k)}$ 是第 k 次QR迭代后的上Hessenberg矩阵。带原点位移的QR迭代法公式如下：

$$\begin{aligned}H^{(k)} - \mu_k I &= Q_k R_k, \\H^{(k+1)} &= R_k Q_k + \mu_k I.\end{aligned}$$

这里的 μ_k 表示位移的量。可以验证这样得到的 $H^{(k+1)}$ 与 $H^{(k)}$ 有相同的特征值，且依然是上Hessenberg矩阵。

接下来考虑如何选取 μ_k 。考虑 $H^{(k)}$ 右下角的2阶矩阵

$$H^{(k)} = \begin{bmatrix} h_{11}^{(k)} & h_{12}^{(k)} & \cdots & h_{1,n-1}^{(k)} & h_{1n}^{(k)} \\ h_{21}^{(k)} & h_{22}^{(k)} & \cdots & h_{2,n-1}^{(k)} & h_{2n}^{(k)} \\ & h_{32}^{(k)} & \ddots & \vdots & \vdots \\ & & \ddots & h_{n-1,n-1}^{(k)} & h_{n-1,n}^{(k)} \\ & & & h_{n,n-1}^{(k)} & h_{nn}^{(k)} \end{bmatrix}$$

带原点位移的QR迭代法

当 k 充分大时, $h_{n,n-1}^{(k)}$ 接近0, $h_{n,n}^{(k)}$ 接近某一个特征值。因此, 可以选取位移

$$\mu_k = h_{n,n}^{(k)}.$$

通过这样选取位移, 下面证明: 如果 $h_{n,n-1}^{(k)} = \varepsilon$ 很小, $H^{(k)} - h_{n,n}^{(k)}I$ 经过一次QR迭代之后, 有 $h_{n,n-1}^{(k+1)} = O(\varepsilon^2)$.

证明: 记 $H^{(k)} - h_{n,n}^{(k)}I$ 的右下角再经过前 $n-2$ 次Givens变换之后为 $\begin{bmatrix} \alpha & \beta \\ \varepsilon & 0 \end{bmatrix}$

$$\text{第 } n-1 \text{ 次 Givens 变换: } \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} \alpha & \beta \\ \varepsilon & 0 \end{bmatrix} = \begin{bmatrix} \sqrt{\alpha^2 + \varepsilon^2} & c\beta \\ 0 & -s\beta \end{bmatrix}, \quad c = \frac{\alpha}{\sqrt{\alpha^2 + \varepsilon^2}}, s = \frac{\varepsilon}{\sqrt{\alpha^2 + \varepsilon^2}}$$

$$\text{于是, } \begin{bmatrix} * & * & * \\ * & h_{n-1,n-1}^{(k+1)} & h_{n-1,n}^{(k+1)} \\ 0 & h_{n,n-1}^{(k+1)} & h_{n,n}^{(k+1)} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & s \\ 0 & -s & c \end{bmatrix} \begin{bmatrix} * & * & * \\ 0 & \alpha & \beta \\ 0 & \varepsilon & 0 \end{bmatrix} \begin{bmatrix} c' & s' & 0 \\ -s' & c' & 0 \\ 0 & 0 & 1 \end{bmatrix}^T \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & s \\ 0 & -s & c \end{bmatrix}^T$$

其中, $\begin{bmatrix} c' & s' \\ -s' & c' \end{bmatrix}$ 是第 $n-2$ 次Givens变换。

$$\text{所以, } \begin{bmatrix} h_{n-1,n-1}^{(k+1)} & h_{n-1,n}^{(k+1)} \\ h_{n,n-1}^{(k+1)} & h_{n,n}^{(k+1)} \end{bmatrix} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} \alpha c' & \beta \\ \varepsilon c' & 0 \end{bmatrix} \begin{bmatrix} c & s \\ -s & c \end{bmatrix}^T + h_{n,n}^{(k)}I = \begin{bmatrix} * & * \\ -s^2\beta & * \end{bmatrix}$$

可见, 通过原点位移, 收敛速度变成了二次收敛。

带原点位移的QR迭代法

当 $h_{n,n-1}^{(k)}$ 收敛到0之后, 如果 $h_{n-1,n-2}^{(k)}$ 还没有收敛到0, 且已知 $h_{n-1,n-1}^{(k)}$ 对应某个实特征值, 那么接下来可以选取位移为 $h_{n-1,n-1}^{(k)}$, 使得 $h_{n-1,n-2}^{(k)}$ 二次收敛到0。

只要 A 的特征值全部为实数, 则可以通过重复以上过程加速得到 A 的全部特征值。

如果 A 有复特征值全部为实数, 那么用实数位移不一定能加速收敛过程。这时, 需要采用**双重步位移的QR迭代**, 具体参见教材6.4.5节, 留给大家自行学习。

由于对称矩阵的特征值必定为实数, 于是下面考虑针对对称矩阵的带位移的QR迭代法。

对称QR方法

对称QR迭代法和QR迭代法原理一致，只是利用了对称矩阵的特性做了一些改进，从而降低了计算量。

大概的计算流程是：

- (1) 上Hessenberg化，即三对角化
- (2) 带位移的QR迭代

三对角化

若 A 是对称矩阵，将 A 上Hessenberg化后可得

$$Q^T A Q = H,$$

其中 H 是上Hessenberg矩阵。易知， H 必是三对角对称矩阵。

因此，在对 A 进行上Hessenberg化时，可以利用三对角的特性降低原有计算方法的计算量。

第一步， H_1 应当选取如下形式：

$$H_1 = \begin{bmatrix} 1 & 0 \\ 0 & \tilde{H}_1 \end{bmatrix}$$

其中 $\tilde{H}_1$ 是 $n-1$ 维的Household变换。可知 $H_1 A H_1^T$ 将第一列和第一行的第 $3 \sim n$ 个元素同时化为0。

三对角化

$$A = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} = \begin{bmatrix} a_{11} & v_0^T \\ v_0 & A_0 \end{bmatrix}$$

$$H_1 A = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ \textcolor{red}{0} & * & * & * \\ \textcolor{red}{0} & * & * & * \end{bmatrix} = \begin{bmatrix} a_{11} & v_0^T \\ \tilde{H}_1 v_0 & \tilde{H}_1 A_0 \end{bmatrix}$$

黑色元素表示和 A 的对应元素一致

$$H_1 A H_1^T = \begin{bmatrix} * & * & \textcolor{red}{0} & \textcolor{red}{0} \\ * & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \end{bmatrix} = \begin{bmatrix} a_{11} & v_0^T \tilde{H}_1 \\ \tilde{H}_1 v_0 & \tilde{H}_1 A_0 \tilde{H}_1 \end{bmatrix}$$

黑色元素表示和 $H_1 A$ 的对应元素一致

$$= \begin{bmatrix} \alpha_1 & \beta_1 & 0 \\ \beta_1 & \alpha_2 & v_1^T \\ 0 & v_1 & A_1 \end{bmatrix}$$

$\alpha_1 = a_{11}$, α_2 是 $\tilde{H}_1 A_0 \tilde{H}_1$ 的 (1, 1) 元素

三对角化

接下来，选取 H_2

$$H_2 = \begin{bmatrix} I_2 & 0 \\ 0 & \tilde{H}_2 \end{bmatrix}$$

其中 $\tilde{H}_2$ 是 $n-2$ 维的Household变换。可知 $H_2 H_1 A H_1^T H_2^T$ 将第二列和第二行的第4~ n 个元素同时化为0。如此重复，经过 $n-2$ 个household变换，使得

$$H_{n-2} \cdots H_1 A H_1 \cdots H_{n-2} = T = \text{一个三对角阵}$$

$$\text{记为 } T = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \end{bmatrix},$$

其中 α_k 是第 k 次迭代之前矩阵的第 k 个对角元素， β_k 是第 k 次household变换时产生的 β 。

三对角化

记 $Q = H_1 \cdots H_{n-2}$, 则

$$Q^T A Q = T.$$

下面利用 A 的对称性简化每次迭代的计算量。

第 k 次迭代时, 设

$$\tilde{H}_k = I - \beta v v^T, \quad v \in \mathbf{R}^{n-k}$$

可知

$$\tilde{H}_k A_{k-1} \tilde{H}_k = A_{k-1} - v w^T - w v^T$$

其中, A_{k-1} 是第 $k-1$ 次迭代后矩阵的右下角子矩阵,

$$w = u - \frac{1}{2} \beta (v^T u) v, \quad u = \beta A_{k-1} v.$$

三对角化算法

```
1  function [A,Q]=mtridiag(A)
2      %将对称矩阵A三对角化, 即 $Q^T A Q$ =三对角
3  -   n=size(A,1);
4  -   Q=eye(n);
5  -   for k=1:n-2
6  -       [v,beta]=house(A(k+1:n,k));
7  -       H=eye(length(v))-beta*(v*v');
8  -       u=beta*A(k+1:n,k+1:n)*v;
9  -       w=u-(beta*v'*u/2)*v;
10 -      A(k+1,k)=norm(A(k+1:n,k),2);
11 -      A(k+2:n,k)=0;
12 -      A(k,k+1)=A(k+1,k);
13 -      A(k,k+2:n)=0;
14 -      A(k+1:n,k+1:n)=A(k+1:n,k+1:n)-v*w'-w*v';
15 -      Q=Q*blkdiag(eye(k),H);
16 -   end
```

带原点位移的对称三对角阵的QR迭代

设 $T^{(0)}$ 是由初始矩阵 A 三对角化后得到的矩阵， $T^{(k)}$ 是第 k 次QR迭代后的三对角矩阵。带原点位移的对称QR迭代公式如下：

$$\begin{aligned}T^{(k)} - \mu_k I &= Q_k R_k, \\T^{(k+1)} &= R_k Q_k + \mu_k I.\end{aligned}$$

这里的 μ_k 表示位移的量。可以验证这样得到的 $T^{(k+1)}$ 与 $T^{(k)}$ 有相同的特征值，且依然是三对角矩阵。

现在考虑 μ_k 的选取。最简单的做法是选取 $\mu_k = T_{n,n}^{(k)}$ ，然而更好的做法是选取 μ_k 为矩阵

$$T^{(k)}(n-1:n, n-1:n) = \begin{bmatrix} \alpha_{n-1} & \beta_{n-1} \\ \beta_{n-1} & \alpha_n \end{bmatrix}$$

的两个特征值中靠近 α_n 的那一个，即

$$\mu_k = \alpha_n + \delta - \operatorname{sgn}(\delta) \sqrt{\delta^2 + \beta_{n-1}^2}, \quad \delta = (\alpha_{n-1} - \alpha_n)/2$$

这就是Wilkinson位移，可达到三次收敛。

三对角阵的QR迭代

由于 T 的特性，对于每一次迭代，不需要像迭代公式那样“先QR分解再逆序相乘”。可通过以下隐含的方式来实现一次迭代。

$$\begin{aligned}
 T^{(k)} - \mu_k I &= \begin{bmatrix} * & * & 0 & 0 \\ * & * & * & 0 \\ 0 & * & * & * \\ 0 & 0 & * & * \end{bmatrix} \xrightarrow{G_{12}T} \begin{bmatrix} * & * & + & 0 \\ 0 & * & * & 0 \\ 0 & * & * & * \\ 0 & 0 & * & * \end{bmatrix} \xrightarrow{G_{12}TG_{12}^T} \begin{bmatrix} * & * & + & 0 \\ * & * & * & 0 \\ + & * & * & * \\ 0 & 0 & * & * \end{bmatrix} \\
 &\xrightarrow{G_{23} \cdots G_{23}^T} \begin{bmatrix} * & * & 0 & 0 \\ * & * & * & + \\ 0 & * & * & * \\ 0 & + & * & * \end{bmatrix} \xrightarrow{G_{34} \cdots G_{34}^T} \begin{bmatrix} * & * & 0 & 0 \\ * & * & * & 0 \\ 0 & * & * & * \\ 0 & 0 & * & * \end{bmatrix} = B
 \end{aligned}$$

再计算 $T^{(k+1)} = B + \mu_k I$. 从而完成一次迭代。

三对角阵的一次QR迭代代码

```
1  function A=tridiag_sym_qrtran(A,m)
2  %A是三对角阵, m是阶数, 进行一次带wilkinson位移的隐式QR迭代
3  d=(A(m-1,m-1)-A(m,m))/2;
4  sgn_d=1;
5  if d<0
6      sgn_d=-1;
7  end
8  u=A(m,m)-A(m,m-1)^2/(d+sgn_d*sqrt(d^2+A(m,m-1)^2));
9  x=A(1,1)-u;
10 z=A(2,1);
11 for k=1:m-1
12     [c,s]=givens(x,z);
13     G=[c s;-s c];
14     A(k:k+1,max(1,k-1):min(m,k+2))=G*A(k:k+1,max(1,k-1):min(m,k+2));
15     A(max(1,k-1):min(m,k+2),k:k+1)=A(max(1,k-1):min(m,k+2),k:k+1)*G';
16     if k<m-1
17         x=A(k+1,k);
18         z=A(k+2,k);
19     end
20 end
21 end
```

注意：上述代码中，并没有“先对 T 做位移，再QR迭代，再位移回来”，只是在做第一步givens变换时，考虑的变换对象是 $(T(1,1) - \mu, T(2,1))^T$ 。这样做可达到同样的目的。其中的原理留给大家自行思考。

注意：该算法只适于不可约对称三对角矩阵，即次对角线均不为0的对称三对角矩阵。

对称QR迭代法代码

```
1 function [iter,D] = sym_qr_eig(A,tol,N)
2 - if nargin < 3
3 -     N = 5000;
4 - end
5 - if nargin < 2
6 -     tol = 1e-8;
7 - end
8 - n = size(A,1); D = zeros(n,1); iter = 0;
9 - m = n; %表示次对角线上, 最后一个非零元素所在行。一开始等于n
10 - A = mtridiag(A);
11 - while iter <= N
12 -     iter = iter + 1;
13 -     l=1;%l表示从第m行沿次对角线往上寻找到的第一个0元素所在行
14 -     for k = m:-1:2 %从第m行沿次对角线往上寻找第一个0元素
15 -         if abs(A(k,k-1)) < tol
16 -             if m-k+1 <= 2
17 -                 la = eig(A(k:m,k:m));
18 -                 D(k:m) = la';
19 -                 m = k-1;
20 -             else
21 -                 l=k; %A(l:m,l:m)表示A中间的大于等于3阶的不可约子矩阵块
22 -                 break;
23 -             end
24 -         end
25 -     end
26 -     if m <= 2
27 -         la = eig(A(1:m,1:m));
28 -         D(1:m) = la';
29 -         break;
30 -     end
31 -     A(l:m,l:m) = tridiag_sym_qrtran(A(l:m,l:m),m-l+1);
32 - end
```

如果将红框中的函数分别替换为
`mhessen`和`hessen_qrtran`, 则该代码
也可以用于求非对称矩阵的特征值。
大家可自行在`hessen_qrtran`基础上多
考虑位移。

求特征向量

由QR迭代法得到矩阵的全部特征值的近似值之后，通常借助带原点位移的反幂法逐个求解更精确的特征值以及相应的特征向量(不论特征值是实数还是复数)。先用反幂法求 T 的特征向量 z ，再通过 $x = Qz$ 得到 A 的特征向量，其中 $Q = H_1 \cdots H_{n-2}$ 。

注意：求特征向量时，我们得到的是三对角阵 T 的特征向量 z ，需要通过 $x = Qz$ 得到 A 的特征向量，其中 $Q = H_1 \cdots H_{n-2}$ ， $Q^T A Q = T$ 。

数值计算方法

数值积分

信息管理与工程学院
冯银波

基本思想

根据积分中值定理，有

$$\int_a^b f(x)dx = f(\xi)(b-a)$$

如果能在 $[a, b]$ 上选取一些节点 $\{x_k\}$ ，然后用 $\{f(x_k)\}$ 的加权平均来代替 $f(\xi)$ ，那么便可以得到积分的近似值。这样的求积公式具有如下形式：

$$\int_a^b f(x)dx = \sum_{k=0}^n A_k f(x_k) + R[f]$$

数值积分的基本思想就是给出一种选取节点 $\{x_k\}$ 和求积系数 $\{A_k\}$ 的方法，使得余项 $R[f]$ 尽可能的小。

复化梯形公式

思路：将区间 $[a, b]$ 分成个小区间，每个区间内用梯形的面积去近似小区间的面积。

复化梯形公式：

(1) 将区间 $[a, b]$ n 等分，取步长 $h = (b - a)/n$ ，分割点记为：

$$x_0 = a,$$

$$x_k = x_0 + kh, \quad k = 0, 1, 2, \dots, n$$

(2) 在区间 $[x_k, x_{k+1}]$ 上，计算梯形面积 I_k ，作为小区间内的积分近似值

$$I_k = \frac{h}{2} [f(x_k) + f(x_{k+1})], \quad k = 0, 1, 2, \dots, n.$$

(3) 对 $\{I_k\}$ 求和（如下），并将其作为积分近似值

$$T_n = \sum_{k=0}^{n-1} I_k = \sum_{k=0}^{n-1} \frac{h}{2} [f(x_k) + f(x_{k+1})] = \frac{h}{2} \left[f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right]$$

复化Simpson公式

思路：在区间 $[x_k, x_{k+1}]$ 上，用抛物线去近似 $f(x)$ ，再去计算小区间积分近似值 I_k

注意：复化梯形公式在每个小区间内相当于用直线去近似 $f(x)$ 。

复化Simpson公式：

(1)略

(2) 在区间 $[x_k, x_{k+1}]$ 上，取 $w_k = \frac{x_k + x_{k+1}}{2}$ ，用通过 $(x_k, f(x_k))$, $(w_k, f(w_k))$,

$(x_{k+1}, f(x_{k+1}))$ 三点的抛物线作为 $f(x)$ 的近似函数，计算相应的积分：

$$\begin{aligned} \text{抛物线方程: } P_2(x) &= \frac{(x - w_k)(x - x_{k+1})}{(x_k - w_k)(x_k - x_{k+1})} f(x_k) \\ &+ \frac{(x - x_k)(x - x_{k+1})}{(w_k - x_k)(w_k - x_{k+1})} f(w_k) \\ &+ \frac{(x - x_k)(x - w_k)}{(x_{k+1} - x_k)(x_{k+1} - w_k)} f(x_{k+1}) \end{aligned}$$

复化Simpson公式

小区间积分近似值:
$$I_k = \int_{x_k}^{x_{k+1}} P_2(x) dx = \frac{h}{6} [f(x_k) + 4f(w_k) + f(x_{k+1})]$$

(3)对 $\{I_k\}$ 求和（如下），并将其作为积分近似值

$$S_n = \sum_{k=0}^{n-1} I_k = \sum_{k=0}^{n-1} \frac{h}{6} [f(x_k) + 4f(w_k) + f(x_{k+1})]$$

类似地，可以构造五点、七点， $\dots$ ， n 点近似函数，在实际运算中，以梯形和Simpson公式为常用。

给定 $f(x)$ 在 $n+1$ 个不同点处的取值，如何构造（不超过） n 次多项式 $P_n(x)$ 通过这 $n+1$ 点，从而可用 $P_n(x)$ 作为 $f(x)$ 的近似函数。

高斯求积公式

求积公式:
$$\int_a^b f(x)dx \approx \sum_{k=0}^n A_k f(x_k)$$

之前的方法都是在节点 $\{x_k\}$ 实现确定好（比如将区间等分）的前提下给出一种计算求积系数 $\{A_k\}$ 的方法。高斯求积公式的思想是：给定节点数目 $n+1$ 的前提下，给出选取 $\{x_k\}$ 以及 $\{A_k\}$ 的方法，它们的选取是依赖于 n 。具体是通过下列方程组

$$\sum_{k=0}^n A_k x_k^l = \int_a^b x^l dx, \quad l = 0, 1, 2, \dots, 2n+1$$

来求解 $2n+2$ 个未知数: $x_0, \dots, x_n, A_0, \dots, A_n$.

如果 $f(x)$ 是一个不高于 $2n+1$ 次的多项式，那么便有 $\int_a^b f(x)dx = \sum_{k=0}^n A_k f(x_k)$ ，即我们直接可以得到积分的精确值。如果 $f(x)$ 近似于一个不高于 $2n+1$ 次的多项式，那么上述方法可以得到一个好的近似值。由于该近似效果依赖于 n 的大小，因此 $\{x_k\}$ 和 $\{A_k\}$ 的选取间接依赖于 $f(\cdot)$ 。

高斯求积公式

在高斯求积公式中引入权函数可以进一步提高计算精确度。具体做法是：令 $f(x) = \frac{g(x)}{\rho(x)}$ ，通过求解方程组

$$\sum_{k=0}^n A_k x_k^l = \int_a^b \rho(x) x^l dx, \quad l = 0, 1, 2, \dots, 2n + 1$$

来确定 $x_0, \dots, x_n, A_0, \dots, A_n$ ，从而得到 $\int_a^b g(x) dx$ 的近似值：

$$\int_a^b g(x) dx = \int_a^b \rho(x) f(x) dx \approx \sum_{k=0}^n A_k f(x_k)$$

这种做法的直观含义：如果 $\frac{g(x)}{\rho(x)}$ 可以近似于一个不高于 $2n + 1$ 次的多项式，那么上述方法可以得到一个 $\int_a^b g(x) dx$ 的好的近似值。

权函数 $\rho(x)$ 需要提前给定，且不能太复杂，否则会造成计算困难。

高斯求积公式

下面介绍求解方程组的方法。

定理： $x_0, \dots, x_n$ 是之前的方程组的解的充要条件是下列 $n + 1$ 次多项式

$$\omega(x) = (x - x_0)(x - x_1) \cdots (x - x_n)$$

与任何次数不超过 n 的多项式 $P(x)$ 带权正交，即

$$\int_a^b \rho(x) P(x) \omega(x) dx = 0.$$

证明：（必要性）已知高斯求积公式可以精确出不高于 $2n + 1$ 次的多项式的积分值。根据 $\omega(x_k) = 0, \forall k$ ，有

$$\int_a^b \rho(x) P(x) \omega(x) dx = \sum_{k=0}^n A_k \rho(x_k) P(x_k) \omega(x_k) = 0.$$

高斯求积公式

(充分性) 现需证明: 存在 $\{A_k\}$ 使得 $\{A_k\}$ 和由定理中条件所确定的 $\{x_k\}$ 是之前的方程组的解, 即由它们确定的高斯求积公式精确计算任意不高于 $2n+1$ 次的多项式的积分。确定 $\{x_k\}$ 之后, 现取 $\{A_k\}$ 为之前的方程组的前 n 个方程的解。下面证明: $\sum_{k=0}^n A_k f(x_k)$ 可以精确计算任意不高于 $2n+1$ 次的多项式 $f(x)$ 的积分。首先, 我们有

$$f(x) = P(x)\omega(x) + q(x)$$

其中 $P(x)$ 和 $q(x)$ 都是不高于 n 次的多项式。由 $\{A_k\}$ 和 $\{x_k\}$ 的获得方式可知, $\sum_{k=0}^n A_k f(x_k)$ 可以精确计算任意不高于 n 次的多项式的积分。所以

$$\int_a^b \rho(x)f(x)dx = \int_a^b \rho(x)q(x)dx = \sum_{k=0}^n A_k \rho(x_k)q(x_k)$$

由 $\omega(x_k) = 0, \forall k$, 知 $f(x_k) = P(x_k)\omega(x_k) + q(x_k) = q(x_k)$ 。因此

$$\int_a^b \rho(x)f(x)dx = \sum_{k=0}^n A_k (x_k)q(x_k) = \sum_{k=0}^n A_k (x_k)f(x_k)$$

证毕。

高斯求积公式

总结：高斯求积公式的确定方法如下，

(1) 策略性地选取一组 $P(x)$ ，(比如： $1, x, x^2, \dots, x^n$ ，但事实上有更好的选法，需要学习正交多项式的相关理论，尤其是三项递推公式)，然后通过下述条件

$$\int_a^b \rho(x) P(x) \omega(x) dx = 0.$$

来构造 $n+1$ 个方程，从而确定 $x_0, \dots, x_n$ 。

(2) 再通过下列线性方程组：

$$\sum_{k=0}^n A_k x_k^l = \int_a^b \rho(x) x^l dx, \quad l = 0, 1, 2, \dots, n-1$$

求确定 $A_0, \dots, A_n$ 。

高斯求积公式

例：当节点数目为2(即 $n = 1$)时，如何确定高斯求积公式

$$\int_0^1 \sqrt{x} f(x) dx \approx A_0 f(x_0) + A_1 f(x_1)$$

中的 x_0, x_1, A_0, A_1 ? (这里 $\rho(x) = \sqrt{x}$)

解：令 $\omega(x) = (x - x_0)(x - x_1) = x^2 + bx + c$. 欲计算 x_0, x_1 ，只需确定 b, c 即可。由上述定理知：

$$\int_0^1 \sqrt{x} \cdot 1 \cdot (x^2 + bx + c) dx = 0, \quad \int_0^1 \sqrt{x} \cdot x \cdot (x^2 + bx + c) dx = 0.$$

通过积分可以得到：

$$\frac{2}{7} + \frac{2}{5}b + \frac{2}{3}c = 0, \quad \frac{2}{9} + \frac{2}{7}b + \frac{2}{5}c = 0.$$

解得 $b = -\frac{10}{9}, c = \frac{5}{21}$. 从而可得到 $x_0 = 0.289949, x_1 = 0.821162$.

高斯求积公式

然后，通过求解下列方程组(即之前的方程组中取 $l = 0, 1$)：

$$A_0 + A_1 = \int_0^1 \sqrt{x} dx = \frac{2}{3}, \quad A_0 x_0 + A_1 x_1 = \int_0^1 \sqrt{x} x dx = \frac{2}{5}$$

可得到 $A_0 = 0.277556, A_1 = 0.389111$ 。

最后，在只取两个点的条件下，我们得到了如下积分近似值：

$$\int_0^1 \sqrt{x} f(x) dx \approx A_0 f(x_0) + A_1 f(x_1)$$

常见的高斯求积公式

给定 $g(x)$ ，当 $\rho(x)$ 取不同形式时，利用上述思路可以得到不同的： $x_0, \dots, x_n, A_0, \dots, A_n$ ，即得到不同的求积公式：

$$\int_a^b g(x) dx = \int_a^b \rho(x) f(x) dx \approx \sum_{k=0}^n A_k f(x_k).$$

$\rho(x)$ 取某些特殊形式时，计算 $x_0, \dots, x_n, A_0, \dots, A_n$ 会变得非常简单。当选取 $\rho(x)$ 时，既要保证 $\frac{g(x)}{\rho(x)}$ 可以近似于一个次数不高的多项式，要保证 $\rho(x)$ 尽可能是简单函数。

取 $\rho(x) = 1$ ，对应的求积公式称为Gauss-Legendre求积公式。

取 $\rho(x) = \frac{1}{\sqrt{1-x^2}}$ ，对应的求积公式称为Gauss-Chebyshev 求积公式。

取 $\rho(x) = e^{-x}$ ，对应的求积公式称为Gauss-Laguerre求积公式。

取 $\rho(x) = e^{-x^2}$ ，对应的求积公式称为Gauss-Hermite求积公式。

Gauss-Kronrod自适应积分步骤

现在要计算 $\int_a^b f(x)dx$ (matlab中采用的权函数是1)

步骤1: 先计算 7 个高斯节点, 按 $\int_a^b f(x)dx \approx \sum_{k=0}^6 A_k f(x_k)$ 得到积分近似值。注意: 这些节点与被积分函数无关, 可以查表直接获得, 通常的计算软件中都自带。也可直接问chatgpt直接获得这些节点的值

步骤2: 新增8个Kronrod节点。原理是: 原来的7个高斯节点可以精确计算不超过13次多项式的积分; 先新增8个点, 共15个点, 由这15个点可以精确计算更高次数(22次)多项式的积分; 新增的节点也与被积分函数无关。可以查表直接获得。

这一步的主要目的: 用以对比7节点和15节点所计算出的积分值是否接近; 如果接近, 那说明当前的积分值已经够精确了; 如果不够精确, 则进入步骤3

步骤3: 将原来的积分区间分成两份, 然后在每个子区间上重新按步骤1、2计算。

作业

1 / 用QR迭代法求解教材第七章上机习题1(2)。

2 / 当节点数目为2(即 $n = 1$)时, 如何确定高斯求积公式

$$\int_{-1}^1 f(x)dx \approx A_0 f(x_0) + A_1 f(x_1)$$

3 / 如果上题中的积分区域改成 $[0, 1]$, 如何快速确定高斯求积公式? (不需要重新计算 x_0, x_1, A_0, A_1)